



ME133A/ME133B: Robotics Kinematics & Path Planning

Caltech ME133A & ME133B Course Projects

Role	Partner final project in both quarters: with a graduate student in ME133a, with a classmate in ME133b
Duration	Two-quarter sequence, ME133a in Fall 2023 and ME133b in Winter 2024
Stack	ME133a: ROS 2, Python, URDF, RViz. ME133b: pure Python and matplotlib
Outcome	Two working final projects, both with real code and demo footage below

This two-quarter sequence develops core theoretical and computational concepts in robotics. Fall quarter (ME133a) is dedicated to understanding the robot mechanism itself: analyzing degrees of freedom, forward/inverse kinematics, and smooth motion generation, effectively looking "inward" at how a robot coordinates its joints. Winter quarter (ME133b) transitions to an "outward" view: planning and navigating around obstacles.

ME133a: An Injured Atlas, Walking on Crutches

My ME133a final project, with a graduate student partner, was a full kinematic simulation in ROS of the Boston Dynamics Atlas robot walking on crutches, specifically simulating an Atlas with an injured right leg keeping weight off that leg while it moves, using two crutches gripped in its hands. I built custom URDF models for the crutches and the choreography that moves the robot through a scripted sequence: gripping the crutches, shifting weight and rotating the torso, then alternating crutch-assisted steps on the good leg while the injured leg is carried through.

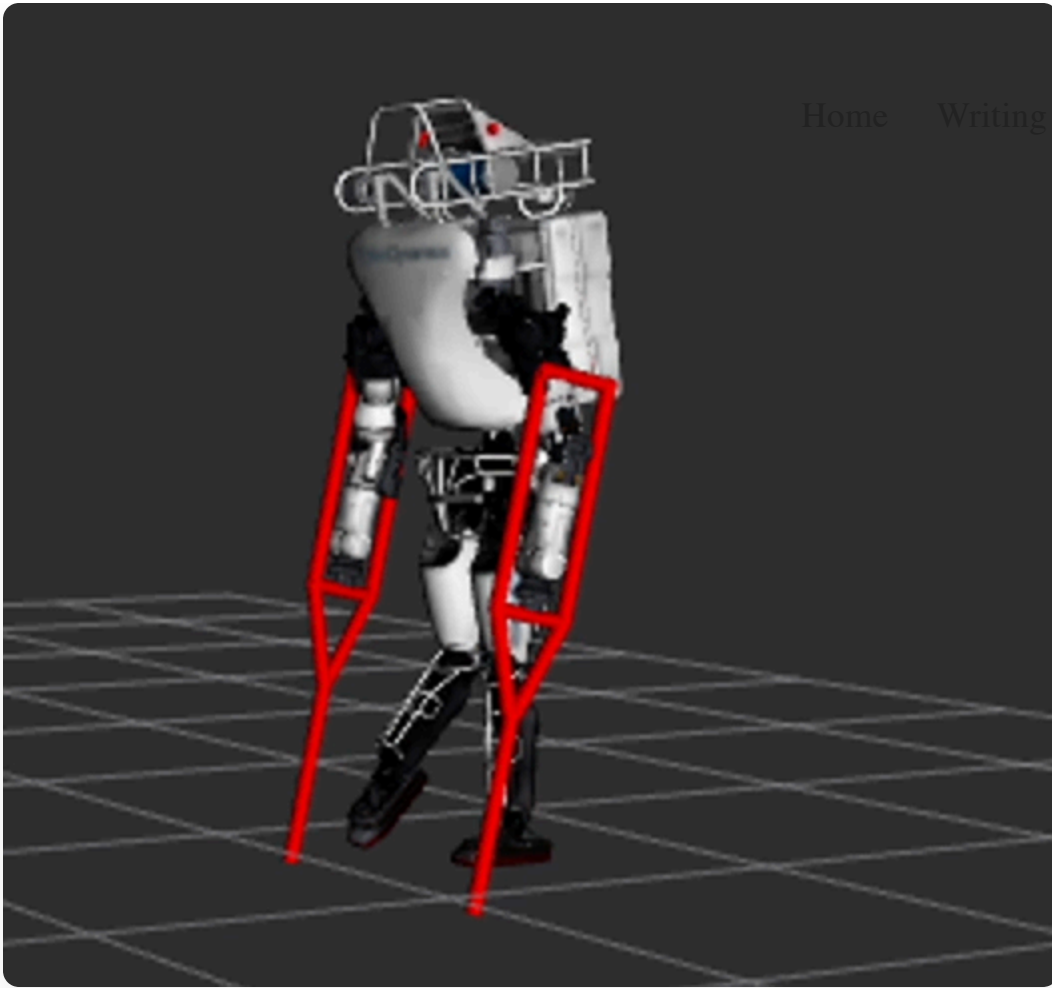
[Home](#)[Writing](#)[Resume](#)

Figure 1: Full kinematic ROS simulation of Atlas walking on crutches with an injured right leg.

The kinematics behind it is what makes this more than a manipulator-arm exercise. Atlas is a 30-DOF floating-base humanoid: the pelvis itself translates and rotates as part of the gait rather than staying fixed in the world, and I solved inverse kinematics independently across six separate kinematic chains hanging off it, both feet, the torso, the head, and both hands, each driven by closed-loop IK through the Jacobian pseudoinverse with proportional error feedback, integrated forward at every timestep. Coordinating six chains and a moving base simultaneously, on a specific timed choreography, is what made this substantially harder than a single fixed-base arm reaching for a target.

[Home](#) [Writing](#) [Resume](#)

1:14 / 1:18

The submitted final demo: Atlas gripping the crutches and walking with its injured leg kept off the ground.

ME133b: Trapezoidal Decomposition and Roadmap Path Planning

My ME133b final project, with a classmate, built a path planner from scratch around trapezoidal cell decomposition: given a 2D world with polygonal obstacles, we decompose the free space into trapezoidal cells, extract each cell's centroid as a roadmap node, connect neighboring centroids, and run A* over that roadmap to find a path between a start and goal point. It's a roadmap-based motion planner built on a real computational-geometry decomposition, not a lookup or a pre-baked grid.

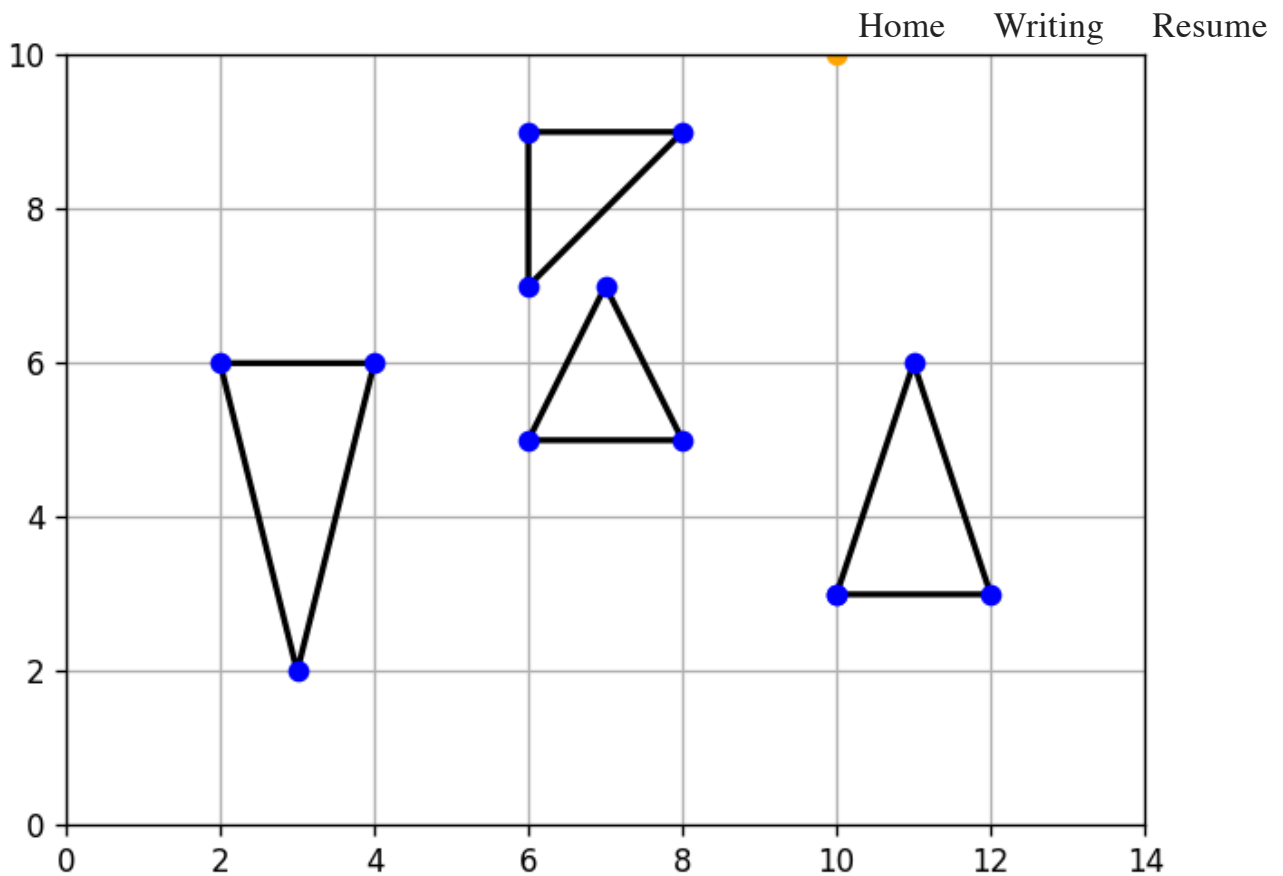


Figure 2: Trapezoidal cell decomposition of a world with triangular obstacles.

Building the decomposition itself was the hard part. Our git history shows real incremental debugging: getting node creation right at the trapezoid vertices, fixing centroid placement, and eventually building the roadmap connectivity, over more than a dozen commits before it produced a usable graph. Trapezoidal

decomposition is notorious for degenerate edge cases at obstacle vertices, and ours has one still: a handful of malformed cells can produce an empty centroid, which needs a small guard to skip cleanly rather than crash the planner.

Home Writing Resume

0:00 / 0:15



The decomposition, roadmap, and resulting A path weaving between all four obstacles.*

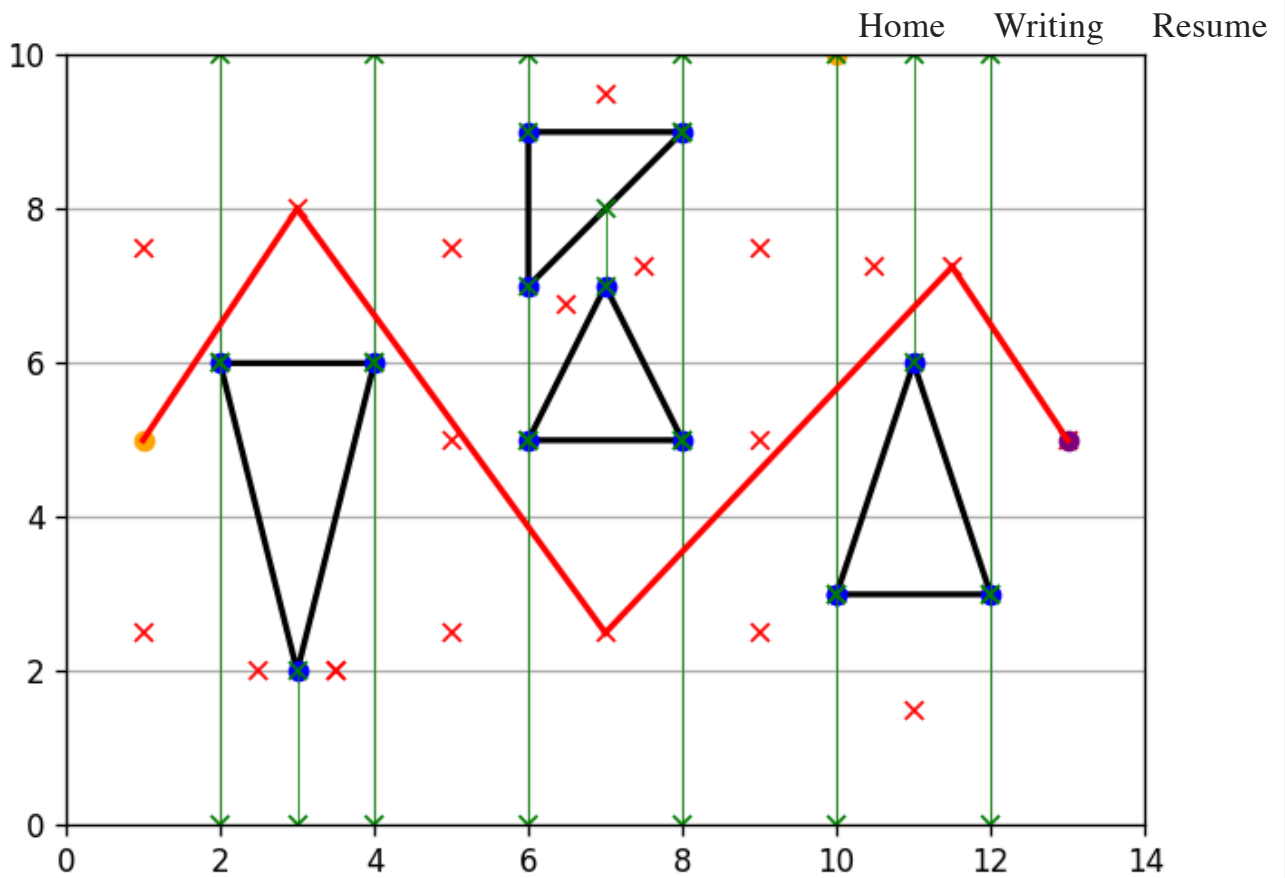


Figure 3: The resulting path, routed through roadmap nodes around every obstacle.

Between the two quarters, ME133a was about coordinating a robot's own many degrees of freedom to hit a target motion, and ME133b was about reasoning over the space a robot moves through to find a path at all. Different halves of the same underlying problem: getting a robot from where it is to where it needs to be.